Diabetes Prediction System

Updated 11 November 2026. Follow the repaired workflow from the bundled CSV to a fitted model that can score a named input record.

1. Start with the data

The CSV contains 768 rows, eight feature columns and the binary Outcome label. There are 500 class-0 rows and 268 class-1 rows. The previous file had an XLS extension even though its contents were CSV. The file was renamed to match its format, and the SHA-256 digest confirms the bytes were preserved.

File	What it does
`diabetes.csv`	Supplies the benchmark records
`diabetes.py`	Validates data, trains, evaluates and scores
`Diabetes.ipynb`	Walks through the same training and prediction functions
`examples/record.json`	Supplies one example with named features
`artifacts/diabetes.joblib`	Stores the fitted pipeline after training

The data uses zero as a missing-measurement code for Glucose, BloodPressure, SkinThickness, Insulin and BMI. Insulin has 374 such values and SkinThickness has 227. A zero pregnancy count has a real meaning and is retained. Dataset reference

> Think of a form where a blank meter reading was entered as zero. Taking the median over those zeros would treat unread measurements as actual readings.

2. Fit preprocessing and the classifier

The loader checks the column names and labels. It converts only the documented measurement codes to missing values. This operation uses a fixed rule, so it can run before splitting.

A stratified 80/20 split with seed 2 assigns 614 rows to training and 154 to the holdout. Stratification keeps the class proportions close. Five-fold validation runs within the 614 training rows. Each fold fits a fresh copy of the entire pipeline.

Median imputation

The imputer learns the median of observed values in each training feature. It fills missing entries using those saved medians. The median is robust to individual extreme observations. The approach compresses variation for imputed rows and can miss informative patterns in which measurements are absent.

Standardisation

The scaler learns each imputed training column's mean and standard deviation. It subtracts the mean and divides by the standard deviation. This keeps columns such as age and insulin from setting the scale of the SVM solely because of their units.

Linear support vector classifier

The linear SVC learns a weighted combination of the scaled features and an intercept. The sign of that margin determines the class, and its magnitude gives the distance from the decision threshold in model-score units. ROC AUC uses this continuous score.

3. Run and reload

```
python -m pip install -r requirements.txt
python diabetes.py train
python diabetes.py predict examples/record.json
```

Training prints JSON and writes the joblib artifact. A later process reloads the saved imputer, scaler and classifier. Inference requires exactly eight named fields, checks numeric validity and reorders the columns before scoring. The supplied record returns class 0 and a score of approximately -1.1125. A valid record has a prediction even when one of the five measurements is missing.

The notebook uses the same Python functions. Install the developer requirements for a notebook kernel and use the repository-root command in the README.

4. Read the holdout result

Actual label	Predicted 0	Predicted 1
0	91	9
1	26	28

Accuracy is 119 / 154 = 77.27%. Positive recall is 28 / 54 = 51.85%. The same majority baseline gets 100 / 154 = 64.94%. With imputation, ROC AUC is 0.8200 on this split. The original run had the same confusion matrix and AUC 0.7919. The scoring order changed, while the total counts at the default threshold stayed the same.

Training-only validation averaged 77.04% accuracy, 72.17% balanced accuracy, 56.09% recall and 0.8451 AUC across five folds. The holdout was already used in the historical audit. Treat the before-and-after comparison as a replay of that split. A different cohort is needed for an independent estimate of generalisation.

Sources: Execution audit | Code and run instructions | Benchmark documentation